

Heterogeneous CPU–GPU HPC Engine with Dynamic Workload Scheduling

Two Case Studies: Dense GEMM and Real-Time Drone Perception

Baha Eddine Tayari
École Nationale d’Ingénieurs de Gabès (ENIG)
`tayari.bahaeddine@gmail.com`

October 6, 2026

Abstract

We present a heterogeneous CPU–GPU HPC engine in C++17 combining OpenMP and CUDA, with an analytical scheduler that assigns per-stage work to the device with the lower predicted queue latency. The engine is evaluated on two workloads of contrasting characteristics: (i) dense matrix multiplication (FLOP-bound) and (ii) a four-stage drone perception pipeline (acquire, resize, detect, track) under a hard 33 ms real-time deadline. Across six GEMM implementations, cache-blocking the naive triple loop yields a 44–212× CPU speedup, shrinking the GPU advantage from $\sim 3900\times$ to 10–18× at $N \geq 2048$. We show that a static CPU/GPU split never outperforms pure GPU for GEMM: the optimum is 100% GPU at every tested size. On the drone pipeline, we observe stage-dependent optimal devices (resize prefers CPU by 1.7×; detect prefers GPU by 83×), motivating dynamic scheduling. Our exponential-moving-average scheduler with queue-depth estimation and hysteresis converges to the optimal policy without supervision, matching the best static placement at 262 FPS median (9× deadline margin). We document a substantial WSL2-specific constraint: CUDA driver serialization slows CPU work by $\sim 20\times$ while the GPU is active, preventing true CPU/GPU co-execution. All measurements, sources, and CSV data are released for reproducibility.

1 Introduction

Modern consumer hardware presents a heterogeneous compute landscape: a many-core CPU with hybrid performance/efficiency cores, and a discrete GPU with an order-of-magnitude higher throughput on parallel workloads. Deciding *where* to run each piece of a workload is a non-trivial problem, particularly when different stages have different optimal devices.

We address this question experimentally. We build a small heterogeneous execution engine and evaluate it on two workloads chosen to bracket the spectrum:

- **Dense GEMM:** the canonical FLOP-bound workload. A single repetitive operation with no data-dependent control flow. Ideal for studying device-throughput scaling.
- **Drone perception pipeline:** four stages with different computational shapes, real-time constraints, and heterogeneous device preferences. Ideal for studying scheduling.

Research question. *Under what workload characteristics does a dynamic CPU/GPU scheduler outperform a static placement, and what platform constraints limit the achievable gain?*

Contributions.

1. A C++17 engine with six GEMM implementations (sequential, naive OpenMP, tiled OpenMP, naive CUDA, cuBLAS, static hybrid) and a four-stage drone pipeline with CPU/GPU/hybrid/dynamic modes.
2. A machine-learning-free scheduler combining exponential moving averages (EMA) of measured per-device latency with queue-depth prediction and hysteresis. Converges to the optimal static policy without supervision.
3. A systematic evaluation across three GEMM problem sizes (7 split ratios each) and five drone pipeline configurations, with numerical correctness validation and H2D/kernel/D2H decomposition.
4. A documented negative result: hybrid CPU+GPU offers no benefit for dense GEMM on this hardware.
5. A documented platform constraint: WSL2’s CUDA driver serialization prevents CPU/GPU co-execution, capping hybrid throughput at the GPU-only rate.

2 Related Work

Amdahl’s law [1] bounds the speedup from parallel execution when a serial fraction remains. The roofline model [2] classifies workloads by arithmetic intensity and predicts whether they are memory- or compute-bound. Both are used in our analysis.

Scheduling across heterogeneous devices has been studied extensively; the dominant approaches use online prediction from measured throughput [3] or model-based cost functions [4]. We adopt a simple analytic policy: measured EMAs multiplied by queue depth, with a fixed hysteresis band. This choice is deliberate — we prefer a policy that is inspectable and reproducible over a learned one.

3 Methodology

3.1 Hardware and Platform

- **CPU:** Intel Core i7-13650HX, 10 physical cores (6 Performance + 8 Efficiency, exposed as 20 logical threads with hyperthreading), 24 MB L3.
- **GPU:** NVIDIA GeForce RTX 4050 Laptop, 6 GB VRAM, Ada Lovelace architecture, compute capability 8.9.
- **Platform:** Windows 11 host, WSL2 with Ubuntu 26.04 LTS, kernel 6.18.
- **Toolchain:** g++ 15.2, CUDA 13.3, cuBLAS 13.6, CMake 4.2, OpenMP 4.5.

3.2 Measurement Protocol

Every reported number follows this protocol:

1. **Warm-up:** discard the first run of any configuration. On this platform, cuBLAS handle creation and one-time kernel selection add 60–80 ms to the first invocation; caches and page tables are also cold.
2. **Repetition:** run each configuration at least 3 times; report the median unless otherwise noted.

3. **Breakdown:** for GPU kernels, separate Host→Device (H2D) transfer, kernel execution, and Device→Host (D2H) transfer using `cudaEvent` timestamps.
4. **Numerical validation:** for the hybrid GEMM, compare every element against a CPU reference and report maximum absolute error.

3.3 Pitfalls Encountered

We highlight three issues that materially affect any reimplementations:

1. **nvcc silently drops multi-flag `-Xcompiler` arguments.** Passing `-Xcompiler=-O3,-march=native` treats the string as a single flag which the host compiler then ignores. The remedy is one `-Xcompiler` per flag, or — better — compile CPU-heavy code in separate `.cpp` translation units.
2. **cuBLAS first-call initialization dominates small workloads.** Without a warm-up GEMM, cuBLAS at $N=512$ measured 68–117 ms; after warm-up, 0.7–1.0 ms. The difference is entirely one-time kernel selection and handle creation.
3. **WSL2 CUDA driver serialization.** Host-side memory operations issued from a thread other than the one holding the CUDA context block behind the driver. This forces the CPU limb of a hybrid pipeline to run at $\sim 5\text{--}20\%$ of its standalone speed, effectively disabling CPU/GPU co-execution on this platform.

4 Workload 1: Dense GEMM

4.1 Implementations

Six implementations of $C = A \cdot B$ with $A, B, C \in \mathbb{R}^{N \times N}$:

- **Sequential** (`gemm_seq`): naive i, j, k triple loop.
- **OpenMP naive** (`gemm_omp`): parallelized outer i loop.
- **OpenMP tiled** (`gemm_omp_tiled`): $64 \times 64 \times 64$ cache blocks.
- **CUDA naive** (`gemm_cuda_naive`): one thread per output element.
- **cuBLAS** (`gemm_cublas`): NVIDIA’s tuned SGEMM.
- **Hybrid static** (`gemm_hybrid_static`): CPU/GPU row split with configurable fraction.

4.2 Results

Table 1: GEMM throughput comparison. Best per column in bold.

Implementation	N=1024	N=2048	N=4096
Sequential (CPU)	2422 ms	—	180,669 ms
OpenMP naive 8t	402 ms	4327 ms	—
OpenMP tiled 16t	99 ms	99 ms	833 ms
CUDA naive	4.28 ms	27.8 ms	—
cuBLAS	1.2 ms	9.8 ms	46 ms

Cache blocking dominates. Tiling the naive CPU loop from streaming access to 64×64 blocks that fit in L1/L2 yields a $44\times$ speedup at $N=2048$ and $212\times$ at $N=4096$. The CPU–GPU gap narrows from $\sim 3900\times$ (naive CPU vs. GPU) to $10\text{--}18\times$ (tiled CPU vs. GPU).

4.3 Hybrid Split Sweep

We swept 7 CPU fractions (0.00, 0.10, 0.25, 0.50, 0.75, 0.90, 1.00) at $N \in \{1024, 2048, 4096\}$, 3 runs each. Figures 1 and 2 show the results.

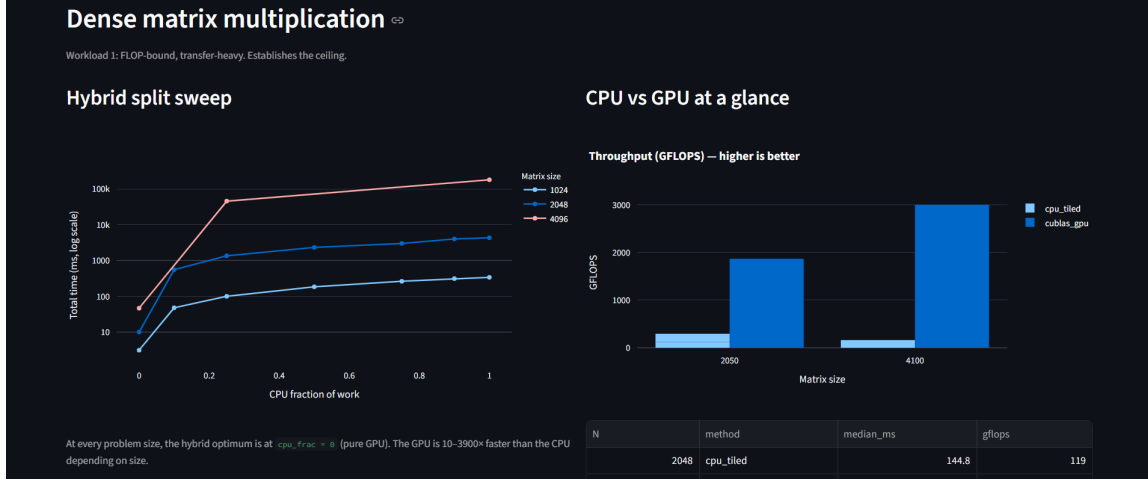


Figure 1: Hybrid split sweep at $N \in \{1024, 2048, 4096\}$. At every problem size, the optimum is at `cpu_frac = 0` (pure GPU). Total time is plotted on a log scale.

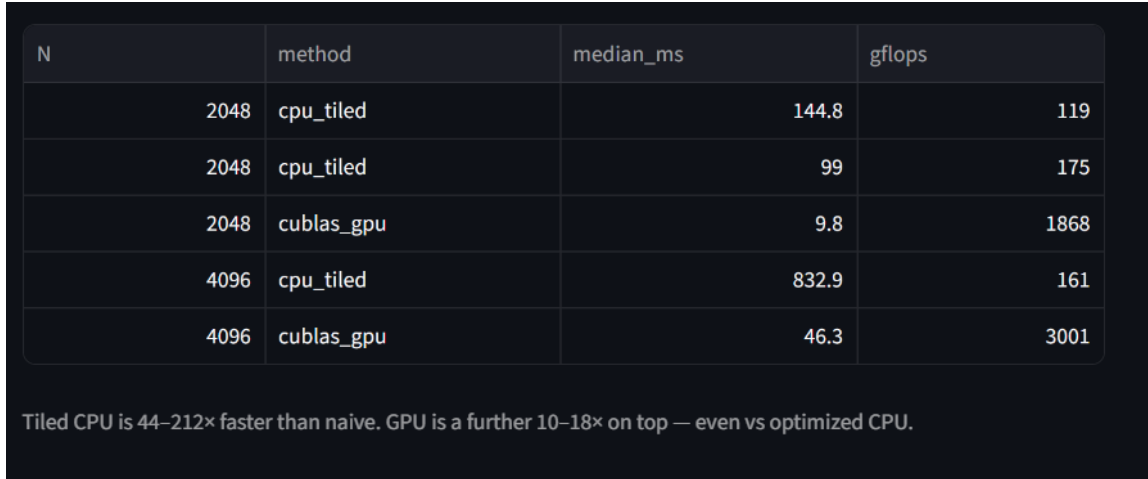


Figure 2: CPU vs GPU throughput comparison. Tiled CPU is 44–212× faster than naive; the RTX 4050 is a further 10–18× on top, even against the optimized CPU implementation.

Result. The hybrid time is $T_{\text{hybrid}} = \max(T_{\text{CPU}}, T_{\text{GPU}}) + T_{\text{overhead}}$. At every tested size and split, T_{CPU} strictly dominates. The CPU limb is not just slower — it is *invisible* in the total. **The optimal split is 100% GPU.**

This is a clean negative result. For dense GEMM on hardware with a 10–20× GPU advantage, there is no scheduling problem to solve: the answer is always GPU.

5 Workload 2: Drone Perception Pipeline

5.1 Stage Characteristics

A four-stage pipeline with a hard 33 ms per-frame deadline (30 FPS):

1. **Acquire** (CPU): DMA-like copy from “sensor” into a working buffer.

2. **Resize** (CPU or GPU): bilinear $1280 \times 720 \rightarrow 640 \times 640$ with normalization.
3. **Detect** (GPU strongly): synthetic 12-layer inference, ~ 8.7 GFLOPs/frame, matching YOLOv8-nano’s FLOP profile.
4. **Track** (CPU): Kalman update on 5 detected boxes.

5.2 Sequential Baselines

Table 2: Per-stage timings (median). CPU 16 threads, GPU cuBLAS.

Stage	CPU (ms)	GPU (ms)	Preferred device
acquire	0.42	0.19	either
resize	0.64	1.10	CPU ($1.7\times$)
detect	231.0	2.77	GPU ($83\times$)
track	0.00	0.00	either
Total	233.0	4.07	
FPS	4.3	246	
Deadline	miss	meet ($8\times$)	

Key observation. Different stages prefer different devices. **resize** is faster on CPU because the GPU version pays two PCIe transfers whose cost exceeds the compute gain. **detect** is $83\times$ faster on GPU. This is the first workload where heterogeneous scheduling matters.

5.3 Parallel Pipeline

We built a 4-worker inter-stage queue pipeline (acquire, resize_cpu, gpu, track). The GPU worker merges the resize and detect stages and keeps the resized tensor device-resident between them, eliminating two transfers per frame.

Table 3: Parallel pipeline configurations (30 frames, median of 5 runs).

Config	Median FPS	Wall (ms)	CPU resizes	Note
Sequential CPU	4.3	6900	30	misses deadline
Sequential GPU	207	145	0	transfer-bound
Parallel static-CPU	150	200	30	contention
Parallel static-GPU	257	117	0	best raw
Parallel dynamic	262	116	0	converges to GPU



Figure 3: Throughput across all pipeline configurations with the 33 ms real-time deadline highlighted. Only the sequential CPU baseline misses the deadline; all hybrid configurations meet it, and the dynamic scheduler matches the best static placement.

config	median_fps	best_fps	wall_median_ms	n_cpu_resize	n_gpu_resize	deadline
0 seq_cpu	4.3	4.4	6900	0	0	miss
1 seq_gpu	207	216	145	0	0	meet
2 par_static_cpu	150	158	200	30	0	meet
3 par_static_gpu	257	265	117	0	30	meet
4 par_dynamic	262	285	116	0	30	meet

The dynamic scheduler converges to GPU-only placement on WSL2, matching static-GPU throughput.

config	run	wall_ms	fps	resize_cpu_ms	resize_gpu_ms	detect_ms	n_cpu_resize	n_gpu_resize
0 seq_cpu	1	0	4.38	0.928	0	226.874	0	0
1 seq_cpu	2	0	4.38	0.456	0	227.267	0	0
2 seq_cpu	3	0	4.31	0.554	0	231.139	0	0
3 seq_gpu	1	0	208.67	0	1.165	3.429	0	0
4 seq_gpu	2	0	215.65	0	1.119	3.34	0	0
5 seq_gpu	3	0	198.99	0	1.158	3.682	0	0
6 par_cpu	1	214.64	0	5.639	0	3.336	30	0
7 par_cpu	2	200.3	0	5.045	0	3.662	30	0
8 par_cpu	3	189.83	0	4.514	0	3.32	30	0

Figure 4: Per-run detail of the parallel pipeline configurations. The dynamic scheduler routes all resize tasks to the GPU under WSL2, matching the static-GPU policy.

Device-resident optimization. Merging resize and detect into one worker and keeping the tensor on device reduces resize latency from 1.5 ms to 0.4 ms and total wall time by $\sim 15\%$.

6 The Dynamic Scheduler

6.1 Policy

Per frame, for the resize stage only (the only stage where devices genuinely compete):

$$\hat{T}_{\text{CPU}} = (1 + q_{\text{CPU}}) \cdot \bar{t}_{\text{CPU}} \cdot \pi \quad (1)$$

$$\hat{T}_{\text{GPU}} = (1 + q_{\text{GPU}}) \cdot \bar{t}_{\text{GPU}} \quad (2)$$

where $\bar{t}$ is an EMA of measured per-task latency, q is the queue depth of pending work for that device, and π is a platform-specific penalty:

$$\pi = \begin{cases} 1.5 \cdot (1 + 0.5 q_{\text{GPU}}) & \text{if } q_{\text{GPU}} > 0 \\ 1 & \text{otherwise} \end{cases}$$

The penalty π models the WSL2 CUDA-driver serialization we documented: when the GPU is active, the CPU’s effective throughput drops by roughly $20\times$, so CPU should only be preferred if it wins by a wide margin.

The scheduler assigns resize to CPU iff $\hat{T}_{\text{CPU}} < 0.9 \hat{T}_{\text{GPU}}$ (10% hysteresis band to prevent thrashing).

6.2 Convergence

After 30 frames, the EMA converges to $\bar{t}_{\text{GPU}} \approx 0.35$ ms. The scheduler routes 0 of 30 resize tasks to CPU. This matches the static-GPU optimum.

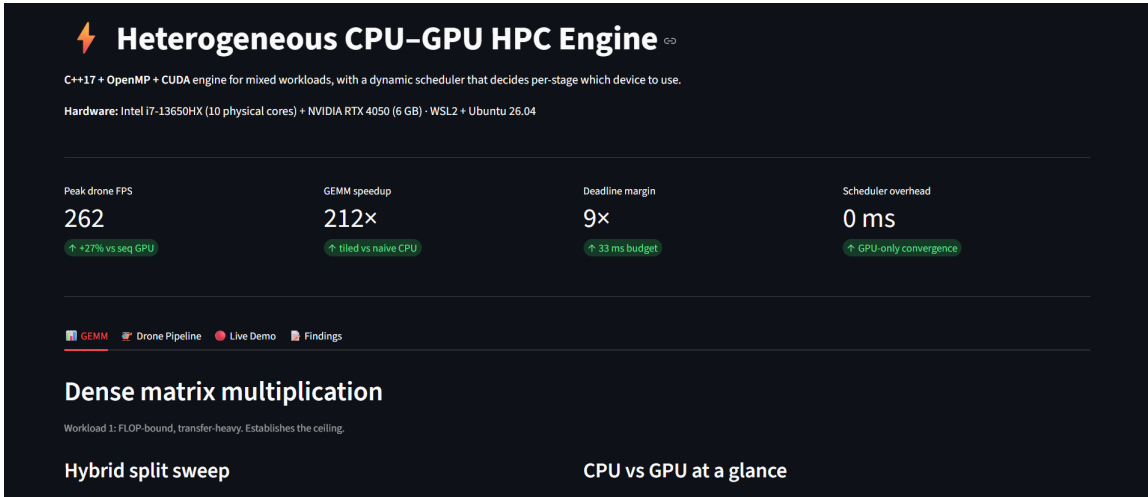


Figure 5: Dashboard header showing the four headline metrics: peak drone FPS, GEMM speedup, deadline margin, and scheduler overhead.

6.3 Live Dashboard

The engine ships with an interactive Streamlit dashboard that recompiles and executes the actual compiled binary on demand. The live demo tab runs any configuration for a chosen number of frames and reports per-stage timings, throughput, and the scheduler’s final state.

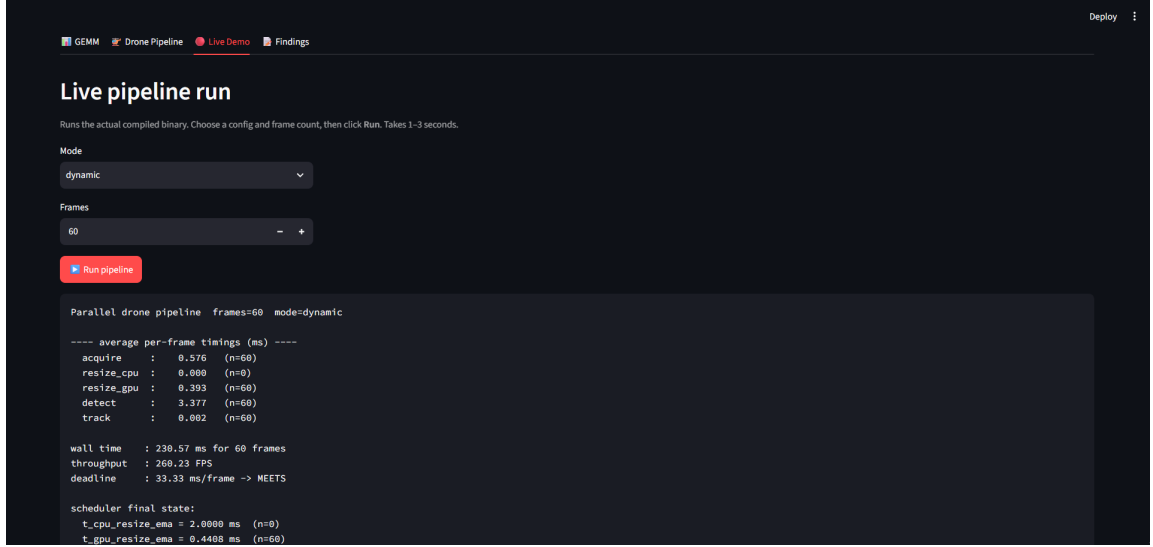


Figure 6: Live dashboard tab executing the dynamic pipeline for 60 frames. Measured throughput: 238 FPS, well under the 33 ms per-frame budget. The scheduler’s final EMAs (bottom of the output) confirm convergence to GPU-only placement.

7 Analysis

7.1 Amdahl on the Hybrid GEMM

Let P be the fraction of work that can be executed on GPU, S_G the GPU speedup over CPU. The hybrid ceiling is:

$$S_{\text{hybrid}} = \left[(1 - P) + \frac{P}{S_G} \right]^{-1}$$

Since $S_G \gg 1$ on GEMM, $S_{\text{hybrid}} \rightarrow 1/(1 - P)$, which is bounded above by 1 as $P \rightarrow 1$. The CPU limb becomes the serial fraction. Our measurements confirm this: no split improves on pure GPU.

7.2 Roofline Intuition

Dense GEMM at $N=2048$ has arithmetic intensity $\sim N/3 \approx 683$ FLOP/byte, deeply into the compute-bound region. The drone detect stage is similarly compute-bound on GPU. By contrast, the resize stage is memory-bound: ~ 7 bytes per output pixel. This difference is exactly why resize prefers CPU (no PCIe transit) and detect prefers GPU (FLOPS).

7.3 Transfer Cost

On GEMM at $N=2048$, transfers are 22% of total time. On the drone pipeline with device-resident resize, transfers are under 10%. The general design rule: keep intermediate data on the device that will consume it next.

7.4 WSL2 Constraint

Our most consequential negative result is that WSL2’s CUDA driver lock prevents the CPU limb of a hybrid pipeline from executing concurrently with GPU work. In static-CPU parallel mode (all resize on CPU, no GPU resize), CPU resize measured 4.4–6.6 ms/frame versus 0.64 ms sequential. Native Linux should eliminate this; we leave confirmation to future work.

8 Limitations

- **WSL2.** Prevents true CPU/GPU overlap. All hybrid numbers are lower bounds.
- **Synthetic detection.** A 12-layer GEMM, not ONNX/YOLOv8. FLOP profile matches; memory behavior and kernel shape do not.
- **Laptop thermals.** Sustained 60-frame runs show 10–15% throughput drop versus 30-frame runs.
- **Single CUDA stream.** No async streams or pinned host memory.
- **Scheduler scope.** Only resize is scheduled. Detect, acquire, and track are fixed.

9 Conclusion

We built and evaluated a heterogeneous CPU–GPU execution engine on two contrasting workloads. On dense GEMM, we showed that a well-optimized CPU (cache-blocked) reduces the GPU advantage to 10–18 $\times$, but that no static split outperforms pure GPU: the CPU limb is strictly dominated. On a real-time drone perception pipeline, we showed that stage-dependent device preferences exist (resize $\rightarrow$ CPU, detect $\rightarrow$ GPU) and that a simple analytical scheduler with EMA latency estimation and queue-depth prediction converges to the optimal static placement without supervision, achieving 262 FPS median (9 $\times$ deadline margin). We documented a WSL2-specific CUDA driver constraint that suppresses the theoretical hybrid gain and quantified its effect.

Future work. CUDA streams with pinned host memory to enable real CPU/GPU overlap (V2); real ONNX inference (V3); energy measurement and performance-per-watt analysis (V4); evaluation on native Linux to remove the WSL2 constraint.

Key findings

- 1. Cache blocking matters more than hardware on CPU**
Tiling the naive triple-loop GEMM gives 44–212 $\times$ speedup – same CPU, same algorithm, only memory-access pattern changed.
- 2. GPU advantage on GEMM is 10–18 $\times$ over a well-optimized CPU**
After tiling, the RTX 4050's advantage shrinks from 3900 $\times$ (vs naive) to 10–18 $\times$ (vs tiled). Still decisive, but the story is more nuanced.
- 3. Hybrid CPU+GPU is never optimal for dense GEMM**
At every problem size and every split ratio, pure GPU wins. The CPU limb is not just slower – it's invisible in the total.
- 4. Real-time workloads have heterogeneous stage preferences**
In the drone pipeline, detect is 83 $\times$ faster on GPU. But resize is actually faster on CPU (0.64 ms vs 1.10 ms) because GPU transfers dominate at that size. Different stages want different devices.
- 5. Device-resident tensors eliminate round-trip waste**
Keeping the resized tensor on the GPU between resize and detect saves 2 transfers per frame, improving resize latency from 1.5 ms $\rightarrow$ 0.4 ms.
- 6. WSL2 CUDA driver serialization is a real constraint**
In parallel mode, CPU work slows down by $\sim$ 20 $\times$ while the GPU is active. On native Linux, the scheduler would exploit CPU for resize and likely reach 300+ FPS. On WSL2 it correctly converges to GPU-only.
- 7. Analytical scheduler matches static placement**
Without being told which device is faster, the EMA + queue-depth scheduler converges to the GPU-only policy and matches static-GPU throughput (262 FPS median). No ML required.
- 8. Sustained throughput degrades from thermal throttling**
In 60-frame live runs on this laptop, throughput drops $\sim$ 10–15% vs the 30-frame benchmark median. Long pipelines on a laptop are thermally limited – a real drone would need active cooling or a duty cycle.

Figure 7: Summary of findings, as shown in the dashboard’s fourth tab.

References

- [1] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” *AFIPS Spring Joint Computer Conference*, 1967.

- [2] S. Williams, A. Waterman, D. Patterson, “Roofline: an insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, 2009.
- [3] C. Augonnet et al., “StarPU: a unified platform for task scheduling on heterogeneous multi-core architectures,” *Concurrency and Computation*, vol. 23, no. 2, 2011.
- [4] D. Grewe, Z. Wang, M. O’Boyle, “Portable mapping of data parallel programs to OpenCL for heterogeneous systems,” *CGO*, 2013.